

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Monitoreo del estado de la batería con BroadcastReceiver en Jetpack Compose				
NÚMERO DE PRÁCTICA:	4	AÑO LECTIVO:	2026-B	NRO. SEMESTRE:	VIII
FECHA DE PRESENTACIÓN	26/09/2026	HORA DE PRESENTACIÓN	23:59		
INTEGRANTE (s): - Zuñiga Villacorta, Peter Sebastian				NOTA (0-20)	
DOCENTE(s): Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS
Github Repo: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab04 1. EJERCICIOS RESUELTOS A continuación se detallan, paso a paso, las instrucciones para mostrar el porcentaje de batería en pantalla usando un BroadcastReceiver registrado con DisposableEffect. Paso 1. Crear un nuevo proyecto con la plantilla "Empty Activity" y crear una función @Composable llamada BatteryScreen(). <pre> 18 @Composable 19 fun BatteryScreen() { 20 // Contenido en los siguientes pasos 21 }</pre> Paso 2. Dentro de BatteryScreen, declarar una variable de estado: var porcentaje by remember { mutableStateOf(0) }. <pre> 18 @Composable 19 fun BatteryScreen() { 20 var porcentaje by remember { mutableStateOf(value = 0) } 21 }</pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

Paso 3. Obtener el contexto actual con `val context = LocalContext.current`, necesario para registrar el receiver.

```
18     @Composable
19     fun BatteryScreen() {
20         var porcentaje by remember { mutableStateOf( value = 0) }
21         val context = LocalContext.current
22     }
```

Paso 4. Dentro de un `DisposableEffect(Unit)`, crear un objeto que extienda `BroadcastReceiver`. En su método `onReceive()`, extraer `EXTRA_LEVEL` y `EXTRA_SCALE` del Intent recibido, calcular el porcentaje ($\text{nivel} * 100 / \text{escala}$) y actualizar la variable de estado.

```
DisposableEffect( key1 = Unit) {
    val receiver = object : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            val nivel = intent?.getIntExtra( name = BatteryManager.EXTRA_LEVEL, defaultValue = -1) ?: -1
            val escala = intent?.getIntExtra( name = BatteryManager.EXTRA_SCALE, defaultValue = -1) ?: -1
            if (nivel != -1 && escala != -1) {
                porcentaje = (nivel * 100) / escala
            }
        }
    }
}
```

Paso 5. Dentro del mismo `DisposableEffect`, registrar el receiver con `context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED))`, e imprimir por consola un mensaje indicando que se registró correctamente.

```
context.registerReceiver( receiver = receiver, filter = IntentFilter( action = Intent.ACTION_BATTERY_CHANGED))
Log.d( tag = "BatteryScreen", msg = "Receiver registrado")
```

Paso 6. Cerrar el `DisposableEffect` con `onDispose { context.unregisterReceiver(receiver) }`, imprimiendo también un mensaje de confirmación por consola, como se resume en la Figura 1.

```
onDispose {
    context.unregisterReceiver( receiver = receiver)
    Log.d( tag = "BatteryScreen", msg = "Receiver desregistrado")
}
```

Paso 7. En el cuerpo de `BatteryScreen`, mostrar el valor con un `Text("Batería: $porcentaje%")`.

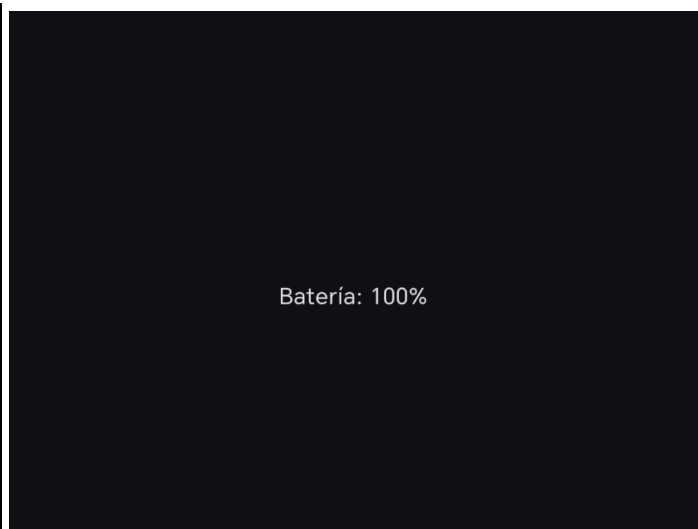
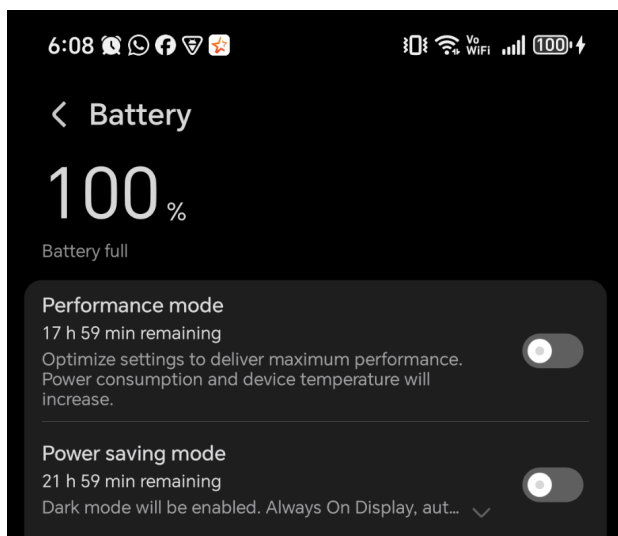
```
Text("Batería: $porcentaje%")
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

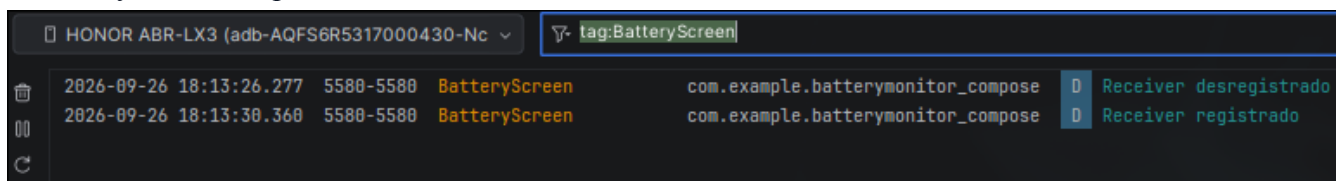
Paso 8. En MainActivity, dentro de setContent {}, reemplazar la llamada a Greeting(...) que trae la plantilla por defecto por BatteryScreen(), para que esta pantalla sea la que se muestra al ejecutar la app.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            BatteryMonitor_ComposeTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Box(
                        modifier = Modifier
                            .fillMaxSize()
                            .padding(paddingValues = innerPadding),
                        contentAlignment = Alignment.Center
                    ) {
                        BatteryScreen()
                    }
                }
            }
        }
    }
}
```

Paso 9. Ejecutar la aplicación en un emulador y, desde Extended Controls > Battery, simular distintos niveles de carga, verificando que el porcentaje mostrado en pantalla se actualiza siguiendo el flujo de la Figura 2.



Paso 10. Revisar Logcat y confirmar que aparecen los mensajes de registro y desregistro del receiver al entrar y salir de la pantalla.



2. EJERCICIOS PROPUESTOS

Tomando como referencia el ejercicio resuelto por el docente, completa la aplicación de inicio de sesión y registro agregando persistencia de datos y una pantalla de bienvenida:

1. **Agrega un botón "Actualizar manualmente" a BatteryScreen. Al presionarlo, en vez de esperar el evento del sistema, dispara un broadcast personalizado: crea un Intent con una acción propia (por ejemplo "com.tuapp.ACTUALIZAR_BATERIA"), envuélvelo en un PendingIntent.getBroadcast(...), y actívalo con PendingIntent.send().**

En BatteryScreen, se agrega un componente Button con el texto "Actualizar manualmente". Al ser presionado, genera un Intent con la acción propia "com.example.batterymonitor_compose.ACTUALIZAR_BATERIA", lo envuelve en un PendingIntent.getBroadcast(...) y lo activa ejecutando send().

```
Button(
    onClick = {
        val customIntent = Intent(ACTION_ACTUALIZAR_BATERIA).apply {
            `package` = context.packageName
        }
        val pendingIntent = PendingIntent.getBroadcast(
            context,
            requestCode = 0,
            customIntent,
            flags = PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
        try {
            pendingIntent.send()
            Log.d( tag = "BatteryScreen", msg = "PendingIntent enviado correctamente")
        } catch (e: PendingIntent.CanceledException) {
            Log.e( tag = "BatteryScreen", msg = "Error al enviar PendingIntent", tr = e)
        }
    }
) {
    Text("Actualizar manualmente")
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

2. Implementa un segundo BroadcastReceiver (o reutiliza el mismo patrón) que escuche esa acción personalizada y, al recibirla, vuelva a leer el estado de la batería (por ejemplo con BatteryManager) y actualice el mismo estado en pantalla.

Se define un segundo BroadcastReceiver que reacciona a la acción personalizada. Al ejecutarse onReceive, lee el estado actual de la batería directamente mediante el sticky intent del sistema (Intent.ACTION_BATTERY_CHANGED), recalcula el porcentaje y actualiza la variable de estado primitiva porcentaje.

```
DisposableEffect( key1 = Unit) {
    val manualReceiver = object : BroadcastReceiver() {
        override fun onReceive(ctx: Context?, intent: Intent?) {
            Log.d( tag = "BatteryScreen", msg = "Broadcast personalizado recibido")

            val batteryStatus: Intent? = ctx?.registerReceiver(
                receiver = null,
                filter = IntentFilter( action = Intent.ACTION_BATTERY_CHANGED),
                flags = Context.RECEIVER_EXPORTED
            )

            val nivel = batteryStatus?.getIntExtra( name = BatteryManager.EXTRA_LEVEL, defaultValue = -1) ?: -1
            val escala = batteryStatus?.getIntExtra( name = BatteryManager.EXTRA_SCALE, defaultValue = -1) ?: -1
            val status = batteryStatus?.getIntExtra( name = BatteryManager.EXTRA_STATUS, defaultValue = -1) ?: -1

            if (nivel != -1 && escala != -1) {
                porcentaje = (nivel * 100) / escala
            }
        }
    }
}
```

3. Registra y desregistra este segundo receiver siguiendo el mismo patrón de DisposableEffect usado en el ejercicio resuelto.

Dado que la aplicación está orientada a API 34+, se registra el receiver especificando explícitamente el flag Context.RECEIVER_NOT_EXPORTED para restringir la recepción de broadcasts únicamente a la propia aplicación. La suscripción y desuscripción se gestionan en el ciclo de vida del composable usando DisposableEffect.

Kotlin

```
DisposableEffect(Unit) {
    val manualReceiver = object : BroadcastReceiver() {
        override fun onReceive(ctx: Context?, intent: Intent?) {
            Log.d("BatteryScreen", "Broadcast personalizado recibido")

            val batteryStatus: Intent? = ctx?.registerReceiver(
                null,
                IntentFilter(Intent.ACTION_BATTERY_CHANGED),
                Context.RECEIVER_EXPORTED
            )
        }
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

-1
val nivel = batteryStatus?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1
val escala = batteryStatus?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1
val status = batteryStatus?.getIntExtra(BatteryManager.EXTRA_STATUS, -1) ?: -1

if (nivel != -1 && escala != -1) {
    porcentaje = (nivel * 100) / escala
}
estaCargando = status == BatteryManager.BATTERY_STATUS_CHARGING ||
               status == BatteryManager.BATTERY_STATUS_FULL
}
}

// Registro directo con bandera obligatoria para API 34+
context.registerReceiver(
    manualReceiver,
    IntentFilter("com.example.batterymonitor_compose.ACTUALIZAR_BATERIA"),
    Context.RECEIVER_NOT_EXPORTED
)
Log.d("BatteryScreen", "Manual Receiver registrado")

onDispose {
    context.unregisterReceiver(manualReceiver)
    Log.d("BatteryScreen", "Manual Receiver desregistrado")
}
}

```

4. Muestra en pantalla, además del porcentaje, si la batería se está cargando o no, usando EXTRA_STATUS del Intent original.

Se optimiza el manejo del estado numérico en Compose mediante mutableIntStateOf(0) para el porcentaje de batería. Se extrae BatteryManager.EXTRA_STATUS del Intent para evaluar si el dispositivo se encuentra cargando (BATTERY_STATUS_CHARGING o BATTERY_STATUS_FULL), almacenándolo en estaCargando y mostrándolo en la UI.

```

val status = batteryStatus?.getIntExtra( name = BatteryManager.EXTRA_STATUS, defaultValue = -1) ?: -1

if (nivel != -1 && escala != -1) {
    porcentaje = (nivel * 100) / escala
}

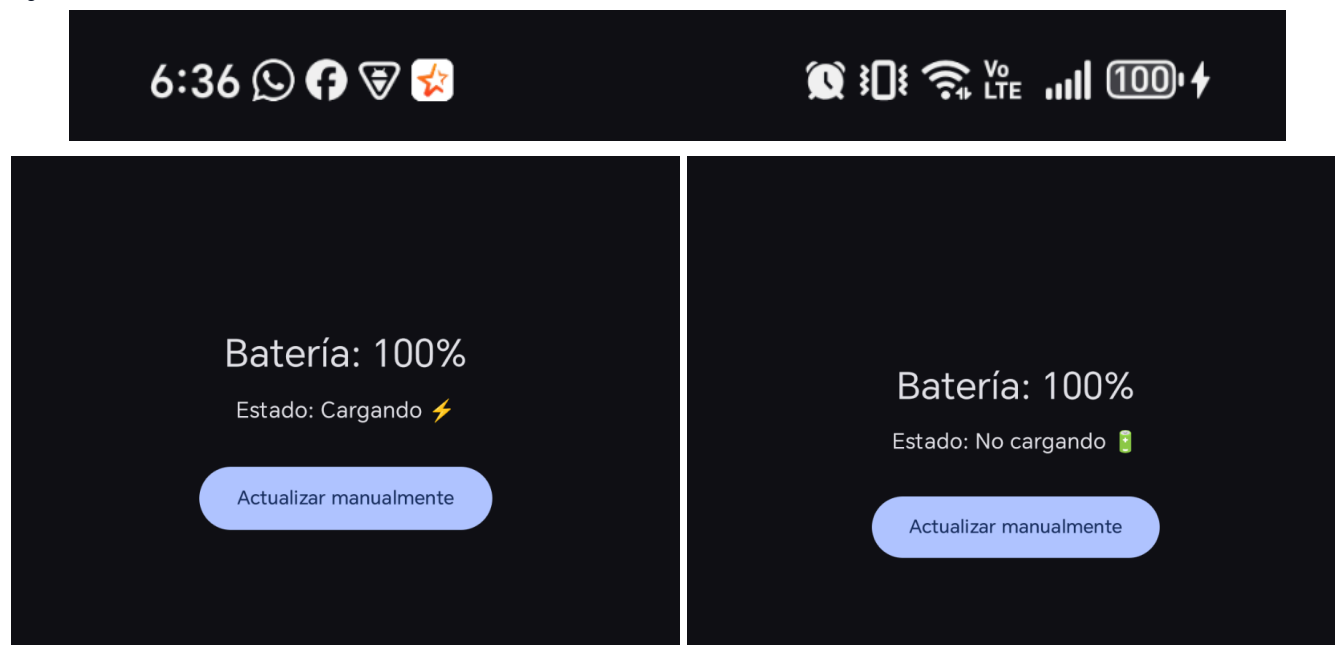
estaCargando = status == BatteryManager.BATTERY_STATUS_CHARGING ||
               status == BatteryManager.BATTERY_STATUS_FULL

Text(
    text = if (estaCargando) "Estado: Cargando 🔌" else "Estado: No cargando 🔋",
    style = MaterialTheme.typography.bodyLarge
)

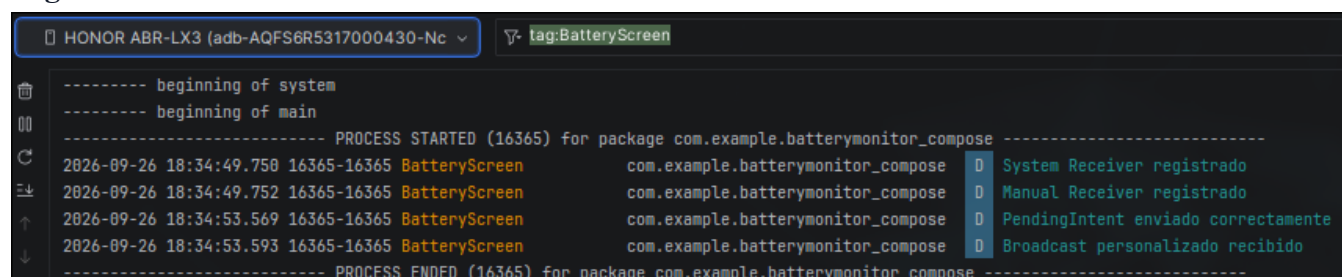
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

Ejecución:



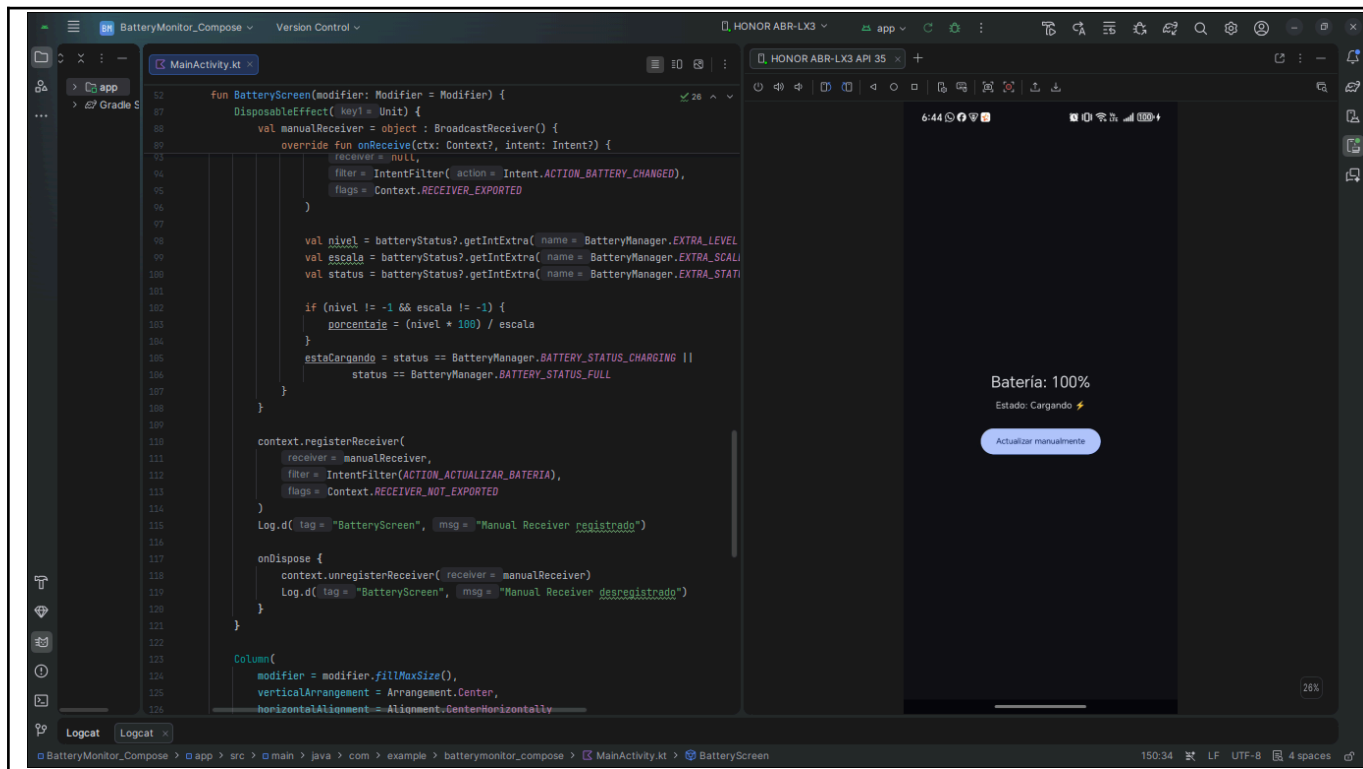
Logcat:



CUESTIONARIO:

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantalla que muestra el porcentaje de batería en tiempo real)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, el ejercicio fue resuelto de manera satisfactoria. Se creó la pantalla BatteryScreen() en Jetpack Compose, integrando un BroadcastReceiver dentro de un DisposableEffect(Unit) para escuchar de forma reactiva la acción Intent.ACTION_BATTERY_CHANGED. El estado del porcentaje de batería se actualiza automáticamente en tiempo real al modificar los valores en los Extended Controls del emulador de Android.



```

102 fun BatteryScreen(modifier: Modifier = Modifier) {
103     DisposableEffect(key1 = Unit) {
104         val manualReceiver = object : BroadcastReceiver() {
105             override fun onReceive(ctx: Context?, intent: Intent?) {
106                 if (intent?.action == Intent.ACTION_BATTERY_CHANGED,
107                     flags = Context.RECEIVER_EXPORTED) {
108                     val nivel = batteryStatus?.getIntExtra(name = BatteryManager.EXTRA_LEVEL,
109                                                         value = 0)
110                     val escala = batteryStatus?.getIntExtra(name = BatteryManager.EXTRA_SCALE,
111                                                            value = 0)
112                     val status = batteryStatus?.getIntExtra(name = BatteryManager.EXTRA_STATUS,
113                                                            value = 0)
114                     if (nivel != -1 && escala != -1) {
115                         porcentaje = (nivel * 100) / escala
116                     }
117                     estaCargando = status == BatteryManager.BATTERY_STATUS_CHARGING ||
118                                     status == BatteryManager.BATTERY_STATUS_FULL
119                 }
120             }
121         }
122         context.registerReceiver(receiver = manualReceiver,
123                                filter = IntentFilter(ACTION_ACTUALIZAR_BATERIA),
124                                flags = Context.RECEIVER_NOT_EXPORTED)
125         Log.d(tag = "BatteryScreen", msg = "Manual Receiver registrado")
126         onDispose {
127             context.unregisterReceiver(receiver = manualReceiver)
128             Log.d(tag = "BatteryScreen", msg = "Manual Receiver desregistrado")
129         }
130     }
131     Column(
132         modifier = modifier.fillMaxSize(),
133         verticalArrangement = Arrangement.Center,
134         horizontalAlignment = Alignment.CenterHorizontally
135     ) {
136         // UI elements
137     }
138 }
    
```



```

----- beginning of system
----- beginning of main
----- PROCESS STARTED (16365) for package com.example.batterymonitor_compose -----
2026-09-26 18:34:49.750 16365-16365 BatteryScreen com.example.batterymonitor_compose D System Receiver registrado
2026-09-26 18:34:49.752 16365-16365 BatteryScreen com.example.batterymonitor_compose D Manual Receiver registrado
2026-09-26 18:34:53.569 16365-16365 BatteryScreen com.example.batterymonitor_compose D PendingIntent enviado correctamente
2026-09-26 18:34:53.593 16365-16365 BatteryScreen com.example.batterymonitor_compose D Broadcast personalizado recibido
----- PROCESS ENDED (16365) for package com.example.batterymonitor_compose -----
    
```

2. En tu propia implementación, ¿qué diferencia encontraste entre recibir el evento automático del sistema (ACTION_BATTERY_CHANGED) y disparar un broadcast manualmente con PendingIntent? ¿En qué situaciones usarías cada uno?

Diferencias Principales:

- **Mecanismo de Disparo y Control:**

- **Broadcast Automático del Sistema (ACTION_BATTERY_CHANGED):** Es un evento reactivo tipo push gestionado internamente por el sistema operativo Android. La aplicación no controla cuándo se emite; se dispara automáticamente cada vez que el hardware detecta una variación en la carga o temperatura de la batería. Además, funciona como un sticky intent (el sistema conserva el último Intent emitido).
- **Broadcast Manual (PendingIntent):** Es un evento explícito iniciado a demanda por el usuario o por un componente específico de la aplicación. Mediante

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

PendingIntent.getBroadcast(...).send(), la aplicación delega la ejecución de un Intent en nombre del usuario o de un proceso externo.

- Flujo de Ejecución:
 - El evento del sistema es continuo e implícito mientras el receiver esté suscrito.
 - El PendingIntent es puntual, controlado y requiere que un desencadenante (como la pulsación de un Button, una notificación o una alarma de AlarmManager) ejecute la acción .send().

Casos de Uso Recomendados:

- Usar ACTION_BATTERY_CHANGED (Sistema) cuando:
 - Se requiere monitoreo pasivo en tiempo real sin intervención del usuario.
 - La interfaz necesita reaccionar automáticamente para ajustar el comportamiento de la app (por ejemplo, reducir el consumo de recursos o cambiar a "Modo Ahorro de Energía" cuando el nivel sea inferior al 15%).
 - Se implementa un Widget en la pantalla de inicio (App Widget) que debe reflejar el nivel de carga actual.
- Usar PendingIntent + Broadcast Personalizado cuando:
 - Se necesita ofrecer una opción de "Refrescar" o "Actualizar a demanda" en la UI.
 - La acción de lectura debe desencadenarse desde fuera del contexto activo de la app, como acciones dentro de Notificaciones del sistema (por ejemplo, botones en una barra de estado).
 - Se programa una tarea diferida mediante AlarmManager o WorkManager que requiere notificar a un BroadcastReceiver en un momento exacto del futuro.

3. En forma individual, comenta en el video de referencia (<https://www.youtube.com/watch?v=AeNnc8yhNTo>) qué otras acciones del sistema, además del cambio de batería, se pueden escuchar con un BroadcastReceiver. (Colocar la captura del comentario; uno por integrante del grupo).

Además del evento ACTION_BATTERY_CHANGED, el sistema Android emite múltiples broadcasts globales a los cuales una aplicación puede suscribirse. Entre las principales acciones destacan:

- Intent.ACTION_POWER_CONNECTED / ACTION_POWER_DISCONNECTED: Notifica en el instante exacto en que el dispositivo se conecta o desconecta de una fuente de energía eléctrica.
- Intent.ACTION_AIRPLANE_MODE_CHANGED: Indica que el usuario ha activado o desactivado el modo avión.
- ConnectivityManager.CONNECTIVITY_ACTION: Detecta cambios en la conectividad de red (alternar entre Wi-Fi, Datos Móviles o pérdida total de red).
- Intent.ACTION_BOOT_COMPLETED: Se emite una sola vez cuando el sistema operativo ha terminado de arrancar completamente. Es muy utilizado para reprogramar alarmas o servicios tras un reinicio del dispositivo.
- Intent.ACTION_SCREEN_ON / ACTION_SCREEN_OFF: Escucha cuando la pantalla del dispositivo se enciende o se apaga.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

- Intent.ACTION_HEADSET_PLUG: Detecta la conexión o desconexión física de audífonos en el conector de audio.

BROADCAST Receivers en Android con Jetpack COMPOSE 📱 La Guía DEFINITIVA 📱



DevExpert - IA para Desarrolladores
63,9 K suscriptores

Suscribirse

👍 105

🗨️

🔗 Compartir

⋮

1,4 K visualizaciones hace 1 año
🔔 No te pierdas ningún contenido y accede a regalos exclusivos apuntándote a la newsletter 📧
<https://listmonk.antoniroleiva.com/sub...> ...más

35 comentarios

≡
Ordenar por


Añade un comentario...



@PETERSEBASTIANZUNIGAVILLACORTA hace 0 segundos

Muy bueno el video, justo me sirvió para una práctica de la universidad. Fue muy importante aprender que los BroadcastReceivers sirven para un montón de eventos del sistema además de la batería, como detectar cuando cambia la red Wi-Fi/datos, cuando la pantalla se apaga/enciende (ACTION_SCREEN_ON/OFF) o cuando se conectan audífonos por cable o Bluetooth. Saludos 🙌

👍 🗨️
Responder

CONCLUSIONES

1. Se comprendió que un BroadcastReceiver es un componente esencial de la arquitectura de Android diseñado para reaccionar ante eventos globales (tanto del sistema como personalizados). Su uso permite que la aplicación responda de manera asíncrona y reactiva a cambios del entorno sin necesidad de mantener procesos pesados en ejecución continua, mejorando la modularidad y eficiencia del software.
2. Se logró implementar exitosamente un BroadcastReceiver orientado a capturar la acción del sistema Intent.ACTION_BATTERY_CHANGED. Mediante la lectura de los extras del Intent (EXTRA_LEVEL, EXTRA_SCALE y EXTRA_STATUS), la aplicación extrae el porcentaje preciso de energía y detecta si el dispositivo está en proceso de carga, actualizando el estado de la aplicación en tiempo real de forma precisa.
3. Se aplicó la función de efecto secundario DisposableEffect para acoplar el registro y desregistro del receiver al ciclo de vida del componente @Composable. Esto asegura que el BroadcastReceiver se suscriba al entrar a la pantalla y se libere automáticamente en la cláusula onDispose al salir de la composición, previniendo fugas de memoria (memory leaks) y evitando consumos innecesarios de batería en el dispositivo.

REFERENCIAS Y BIBLIOGRAFÍA

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

- [1] Android Developers, "Descripción general de broadcasts". [En línea]. Disponible: <https://developer.android.com/develop/background-work/background-tasks/broadcasts>
- [2] Android Developers, "Battery monitoring". [En línea]. Disponible: <https://developer.android.com/training/monitoring-device-state/battery-monitoring>
- [3] Android Developers, "Side-effects in Compose". [En línea]. Disponible: <https://developer.android.com/develop/ui/compose/side-effects>
- [4] "Broadcast Receivers en Android con Jetpack Compose", video. [En línea]. Disponible: <https://www.youtube.com/watch?v=AeNnc8yhNTo>